CVA Dashboard: AI-assisted SDLC

Notion AI inputs (Module 6)

Source notes to paste into a Notion page before prompting Notion AI. Every line is lifted from the repo; nothing here is new. Repo: https://github.com/bleunguts/claude-ai-sdlc-cva-dashboard

Project

- Small CVA (credit valuation adjustment) monitoring dashboard: one filterable table of CVA charge by counterparty (20 mock rows: counterparty, CDS spread, tenor, CVA charge, timestamp).
- Purpose: a vehicle to use one AI tool per SDLC stage and record why. Not a pricing engine. CDS spreads and CVA charges are static mock values, not computed.
- Two narrative threads: (a) real CVA domain depth (a full-stack CVA UI was previously built in React/C#); (b) upskilling via AI-assisted development, shown by rebuilding the equivalent surface through this pipeline.
- Target aesthetic: Bloomberg/AG Grid terminal density, not consumer SaaS.

Stack (from `app/artifacts/cva-monitor/package.json`)

React + TypeScript, Vite, Tailwind, Storybook (`storybook`, `@storybook/react-vite`), Playwright (`@playwright/test`). Data: `design/mock-data.json`. Runs locally with plain npm; no backend is used.

Pipeline and status

#	Stage	Tool	Output	Status
1	Design	Stitch (compared with	Light and dark mockups, tool	Done

#	Stage	Tool	Output	Status
		AI Studio, Figma Make)	comparison in `design/README.md`	
2	Build	Replit Agent	One-page React app in `app/`, 3 prompts	Done
3	Component testing	Storybook	`CvaGrid` Empty / Populated / Error stories, screenshots in `design/`	Done
4	E2E testing	Playwright	1 passing test: filter on "group", 4 rows present, BNP Paribas absent	Done
5	Review	CodeRabbit	Planted edge in PR #21; CodeRabbit never reviewed it	Blocked
6	Docs	Notion AI	This one-pager	In progress

Why this tool, and what happened

1 Stitch. Chosen to generate dense, terminal-style designs fast. Prompt that worked best asked for less: removed nav, KPI cards and extra columns; fixed row height 26px, monospace right-aligned numerics. AI Studio gave the quickest design-to-code export. Figma Make drifted on typography and density (serif font, lost monospace, 8 paginated rows). Decision: light variant is the build target; dark is reference only.

2 Replit Agent. Chosen for spec-to-code with a controlled structure. Prompt 1 gave a working UI that followed the spec, but the scaffold was a pnpm monorepo with an unused Express api-server, Postgres/Drizzle and OpenAPI packages: "no backend" constrained the app, not the scaffold. Prompt 2 (dark theme) was a detour that contradicted the Module 1 decision. Prompt 3 (theme-only, explicit hex palette, "do not change layout, features, data, filters, sorting or `CvaGrid` props") produced a three-file diff. Lesson: name exactly what to change and what not to touch. AI Studio vs Replit: design-export tool for pixels fast, agent for structure

and the dataset you chose; neither removed the cleanup (AI Studio: delete scope; Replit: delete scaffold).

3 Storybook. Chosen to isolate `CvaGrid`. Stories written by hand with Claude Code as reviewer. `storybook init` over-installed (Vitest addon, Playwright provider, Chromatic, MCP addons, rewrote `vite.config.ts`); pruned back to two packages. Error state uses the existing `ErrorBoundary` as a decorator instead of adding an error prop. A green build did not prove the story rendered; screenshots are the evidence. Known rough edge: fallback uses `min-h-screen`.

4 Playwright. Chosen for the user-visible behaviour (filter narrows the table). Test typed by hand with Claude Code coaching. My hand count (3 rows) was wrong; the test received 4 because "Citigroup" contains "group". Expectations should come from the data. Asserting absence (`toHaveCount(0)`) makes it a filter test rather than a render test. A deliberate break (`matchesCounterparty = true`) confirmed the test can fail. Final run: 1 passed (5.9s). Known gaps: one test, one browser, no CI.

5 CodeRabbit (blocked). Planted an unmemoised `filteredRows` in `App.tsx` on PR #21. CodeRabbit never reviewed: tried install, reinstall, reopen, `@coderabbitai review`. Ruled out private repo, draft PR, config file, repo selection. Webhook deliveries are owner-only so could not be inspected. Parked, issue #16 and PR #21 left open. No review results exist, so none may be claimed. Lessons: third-party installs fail silently; verify the bot responds on a throwaway PR before planting the edge.

What I'd change next time (candidate points, from the retros)

- Check each prompt against recorded decisions before running it (Module 2 dark-theme detour).
- Prune or constrain tool defaults early: scaffolds and generators over-deliver.
- Verify third-party integrations on a throwaway PR first (Module 5).
- Derive test expectations from the data, and prove tests can fail.
- Open the rendered output; a green build is not evidence.

Links to cite

- Design: `design/README.md`, `design/UI-stitch.jpg`

- Build log and retros: `app/PROMPTS.md`
- Storybook stories: `app/artifacts/cva-monitor/src/CvaGrid.stories.tsx`; screenshots `design/storybook-*.jpg`
- E2E test: `app/artifacts/cva-monitor/e2e/cva-filter.spec.ts`
- Blocked review: PR #21, issue #16

Notion AI starter prompt

> From these notes, write a one-page summary: the problem (monitoring CVA by counterparty), the approach (one AI tool per SDLC stage), the stack, and what I'd change next time. Factual and concise. CDS spreads are mock inputs; don't claim a pricing engine.

Starter Page